EXP 1:

Aim: Print different patterns of asterisk (*) using loops (e.g. triangle of *).

```
class Pattern3 {
    public static void main(String[] args) {
        int n = 5;

        for (int i = 1; i <= n; i++) {

            // Print spaces
            for (int j = i; j < n; j++) {
                System.out.print(" ");
            }

            // Print stars
            for (int j = 1; j <= (2 * i - 1); j++) {
                System.out.print("*");
            }

            // Move to next line
            System.out.println();
        }
    }
}
```

Exp 2:

Aim: Build a class Employee which contains details about the employee and compile and run its instance.

```
class Employee {
   private int id;
   private String name;
   private String designation;
   private double salary;
   // Constructor
   Employee(int id, String name, String designation, double salary) {
      this.id = id;
      this.name = name;
      this.designation = designation;
      this.salary = salary;
   }
   // Method to display employee details
   void display() {
      System.out.println("Employee Details:");
      System.out.println("ID: " + id);
      System.out.println("Name: " + name);
      System.out.println("Designation: " + designation);
      System.out.println("Salary: " + salary);
   }
   // Main method
   public static void main(String[] args) {
      // Creating an Employee object
      Employee e = new Employee(101, "Ram Jadhav", "Software Engineer", 60000);

      // Displaying employee details
      e.display();
   }}
```

EXP 3: Aim: Write a Java program to create a Book class with attributes title, author, and price. Define a parameterized constructor to initialize these attributes. Create two objects of the class and display their details.

```
class Book {
   private String title;
   private String author;
   private double price;
   Book(String t, String a, double p) {
      title = t;
      author = a;
      price = p;
   }
   void display() {
      System.out.println("Book Details:");
      System.out.println("Title : " + title);
      System.out.println("Author : " + author);
      System.out.println("Price : " + price);
      System.out.println("-------------------");
   }
   public static void main(String[] args) {
      Book b1 = new Book("Let Us Java", "Yashavant Kanetkar", 399.00);
      Book b2 = new Book("The Alchemist", "Paulo Coelho", 299.50);

      b1.display();
      b2.display();
   }
}
```

EXP 4: Aim: Create a class Employee and encapsulate the data members.

```
class Employee {
    private int empId;
    private String name;
    private double salary;

    Employee(int empId, String name, double salary) {
        this.empId = empId;
        this.name = name;
        this.salary = salary;
    }

    public int getEmpId() {
        return empId;
    }

    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
```

```
    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    void display() {
        System.out.println("Employee ID: " + empId);
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
    }

    public static void main(String[] args) {
        Employee emp = new Employee(101, "John Doe", 50000);
        emp.display();
        emp.setSalary(55000);
        System.out.println("Updated Salary: " + emp.getSalary());
    }
}
```

EXP 5: Aim: Print default values of static & instance variables for different data types.

```
class DefaultValues {
   int a;
   double b;
   boolean c;
   char d;
   String e;
   static int sa;
   static double sb;
   static boolean sc;
   static char sd;
   static String se;
   void displayInstanceVal() {
      System.out.println("Instance Variables:");
      System.out.println("int: " + a);
      System.out.println("double: " + b);
      System.out.println("boolean: " + c);
      System.out.println("char: " + d);
      System.out.println("String: " + e);
   }
   static void displayStaticVal() {
      System.out.println("Static Variables:");
      System.out.println("int: " + sa);
      System.out.println("double: " + sb);
      System.out.println("boolean: " + sc);
      System.out.println("char: " + sd);
      System.out.println("String: " + se);
   }
   public static void main(String[] args) {
      DefaultValues obj = new DefaultValues();
      obj.displayInstanceVal();
      displayStaticVal();
   }
}
```

Exp 6.1: Aim: Create sample classes to understand boxing & unboxing.

```
class WrapperExample1 {
    public static void main(String args[]) {
        int a = 20;

        Integer i = Integer.valueOf(a);
        Integer j = a;

        System.out.println(a + " " + i + " " + j);
    }
}
```

6.2:

```
class WrapperExample2 {
    public static void main(String args[]) {
        Integer a = Integer.valueOf(3);
        int i = a.intValue();
        int j = a;

        System.out.println(a + " " + i + " " + j);
    }
}
```

EXP 7: Aim: Create demo applications to illustrate different types of inheritance. (name InheritanceDemo)

```
class Vehicle{
   String brand;
   int speed;
   public Vehicle(String brand, int speed) {
      this.brand = brand;
      this.speed = speed;
   }
   public void showDetails() {
      System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
   }
}
class Car extends Vehicle {
   int doors;
   public Car(String brand, int speed, int doors) {
      super(brand, speed);
      this.doors = doors;
   }
   public void showCarDetails() {
      showDetails();
      System.out.println("Doors: " + doors);
   }
}
class ElectricCar extends Car {
   int batteryCapacity;
   public ElectricCar(String brand, int speed, int doors, int batteryCapacity) {
      super(brand, speed, doors);
      this.batteryCapacity = batteryCapacity;
   }
   public void showElectricCarDetails() {
```

```
        showCarDetails();
        System.out.println("Battery Capacity: " + batteryCapacity + " kWh");
    }
}
class Bike extends Vehicle {
    String type;
    public Bike(String brand, int speed, String type) {
        super(brand, speed);
        this.type = type;
    }
    public void showBikeDetails() {
        showDetails();
        System.out.println("Type: " + type);
    }
}
class InheritanceDemo {
    public static void main(String[] args) {
        System.out.println("--- Single Inheritance ---");
        Car car = new Car("Toyota", 180, 4);
        car.showCarDetails();
        System.out.println("\n--- Multilevel Inheritance ---");
        ElectricCar eCar = new ElectricCar("Tesla", 200, 4, 100);
        eCar.showElectricCarDetails();

        System.out.println("\n--- Hierarchical Inheritance ---");
        Bike bike = new Bike("Yamaha", 120, "Sport");
        bike.showBikeDetails();
    }
}
```

EXP 8: To implement multilevel inheritance in Java using different packages

8.1: package users;

```
public class User {
   protected String name;
   protected String email;

   public User(String name, String email) {
      this.name = name;
      this.email = email;
   }

   public void showDetails() {
      System.out.println("Name: " + name);
      System.out.println("Email: " + email);
   }
}
```

8.2:

```
package customers;
import users.User;
public class Customer extends User {
   protected String address;
   public Customer(String name, String email, String address) {
      super(name, email);
      this.address = address;
   }
   public void showCustomerDetails() {
      showDetails();
      System.out.println("Address: " + address);
   }
}
```

8.3:

```
package primecustomers;

import customers.Customer;

public class PrimeCustomer extends Customer {

    private String membershipType;

    public PrimeCustomer(String name, String email, String address, String membershipType) {

        super(name, email, address);

        this.membershipType = membershipType;

    }

    public void showPrimeCustomerDetails() {

        showCustomerDetails();

        System.out.println("Membership Type: " + membershipType);

    }

}
```

8.4:

```
import primecustomers.PrimeCustomer;


public class MultilevelInheritanceTest {

    public static void main(String[] args) {

        PrimeCustomer primeUser = new PrimeCustomer("Alice", "alice@example.com", "123 Street, NY", "Gold");

        primeUser.showPrimeCustomerDetails();

    }

}
```

Make 3 Folders to run:

users/User.java
customers/Customer.java
primecustomers/PrimeCustomer.java
MultilevelInheritanceTest.java

to run : javac users/User.java customers/Customer.java primecustomers/PrimeCustomer.java MultilevelInheritanceTest.java

java MultilevelInheritanceTest

EXP 9:

Aim: Create an Array of Employee class and initialize array elements with different employee objects. **Save name as EmployeeArray.java**

```
import java.util.Scanner;

class Employee {

   int id;

   String name;

   String designation;

   double salary;

   public Employee(int id, String name, String designation, double salary) {

      this.id = id;

      this.name = name;

      this.designation = designation;

      this.salary = salary;

   }

   public void display() {

      System.out.println("ID: " + id + ", Name: " + name + ", Designation: " + designation + ", Salary: " + salary);

   }

}

public class EmployeeArray {

   public static void main(String[] args) {

      Scanner scanner = new Scanner(System.in);

      System.out.print("Enter the number of employees: ");

      int numEmployees = scanner.nextInt();

      scanner.nextLine();


      Employee[] employees = new Employee[numEmployees];

      for (int i = 0; i < numEmployees; i++) {

         System.out.println("\nEnter details for Employee " + (i + 1));

         System.out.print("Enter Employee ID: ");

         int id = scanner.nextInt();
```

```
            scanner.nextLine();
            System.out.print("Enter Employee Name: ");
            String name = scanner.nextLine();
            System.out.print("Enter Employee Designation: ");
            String designation = scanner.nextLine();

            System.out.print("Enter Employee Salary: ");
            double salary = scanner.nextDouble();
            scanner.nextLine();
            employees[i] = new Employee(id, name, designation, salary);
        }
        System.out.println("\nEmployee Details:");
        for (Employee emp : employees) {
            emp.display();
        }
        scanner.close();
    }
}
```

Exp 10:

Aim: Create a demo application to understand the role of access modifiers

```
class BankAccount {
   private String accountHolderName;
   private double balance;
   public BankAccount(String accountHolderName, double initialBalance) {
      this.accountHolderName = accountHolderName;
      this.balance = initialBalance;
   }
   public void deposit(double amount) {
      if (amount > 0) {
         balance += amount;
         System.out.println("Deposit successful. New balance: " + balance);
      }
   }
   public double getBalance() {
      return balance;
   }
}
class AccessModifierDemo {
   public static void main(String[] args) {
      BankAccount account = new BankAccount("Alice", 5000);
      account.deposit(2000);
      System.out.println("Current Balance: " + account.getBalance());
   }
}
```

Save name as AccessModifierDemo

EXP 11:

Aim: Design a vehicle class hierarchy in Java, and develop a program to demonstrate Polymorphism.

```
import java.util.Scanner;

class Vehicle {

   void start() {

      System.out.println("The vehicle is starting.");

   }

}

class Car extends Vehicle {

   @Override

   void start() {

      System.out.println("The car starts with a key or push button.");

   }

}

class Bike extends Vehicle {

   @Override

   void start() {

      System.out.println("The bike starts with a self-start or kick.");

   }

}

class Truck extends Vehicle {

   @Override

   void start() {

      System.out.println("The truck starts with a heavy-duty ignition system.");

   }

}

public class VehicleDemo {

   public static void main(String[] args) {

      Scanner scanner = new Scanner(System.in);

      System.out.println("Choose a vehicle type to start:");
```

```
        System.out.println("1. Car");
        System.out.println("2. Bike");
        System.out.println("3. Truck");
        System.out.print("Enter your choice (1-3): ");
        int choice = scanner.nextInt();
        Vehicle myVehicle;
        switch (choice) {
            case 1:
                myVehicle = new Car();
                break;
            case 2:
                myVehicle = new Bike();
                break;
            case 3:
                myVehicle = new Truck();
                break;
            default:
                System.out.println("Invalid choice! Defaulting to a generic vehicle.");
                myVehicle = new Vehicle();
        }
        myVehicle.start();
        scanner.close();
    }
}
```

SAVE NAME AS VehicleDemo.java

EXP 12:

Aim: Implement wait,notify,and notifyAll methods. **SAVE FILE NAME AS WaitNotifyDemo.java**

```
class Message {
    private String content;
    private boolean available = false;
    public synchronized void produce(String msg) {
        while (available) {
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        content = msg;
        available = true;
        System.out.println("Produced: " + msg);
        notify();
    }
    public synchronized void consume() {
        while (!available) {
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("Consumed: " + content);
        available = false;
        notifyAll();
    }
}
```

```
class Producer extends Thread {
   private Message msg;
   Producer(Message m) {
      msg = m;
   }
   public void run() {
      for (int i = 1; i <= 5; i++) {
         msg.produce("Message " + i);
      }
   }
}
class Consumer extends Thread {
   private Message msg;
   Consumer(Message m) {
      msg = m;
   }
   public void run() {
      for (int i = 1; i <= 5; i++) {
         msg.consume();
      }
   }
}
public class WaitNotifyDemo {
   public static void main(String[] args) {
      Message m = new Message();
      Producer p = new Producer(m);
      Consumer c = new Consumer(m);
      p.start();
      c.start();
   } }
```